FG4017 DEVLOG

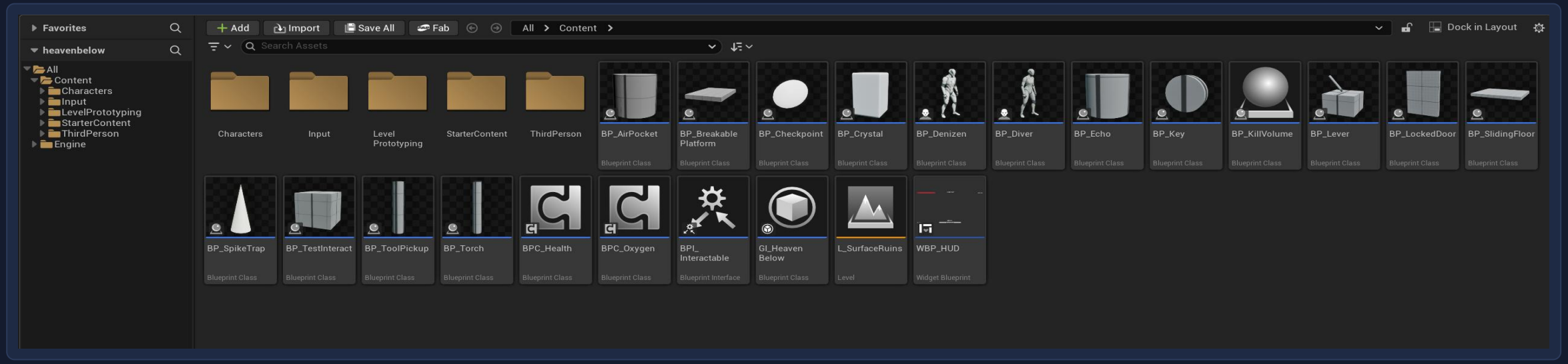
Development Log

Bruno Mirone BA (Hons) Game Design, Level 4, UCA

Built in Unreal Engine 5.6 using Blueprints only

Everything in the project

Twenty two assets in total: sixteen actors, two shared components, one interface, a Game Instance, one widget and one level.



Content Browser

The level

Level 1 is one enclosed vertical shaft with six landings that alternate left and right, so all progress is downward and there is no way back up.

L0 Spike trap, then a breakable platform that drops you in

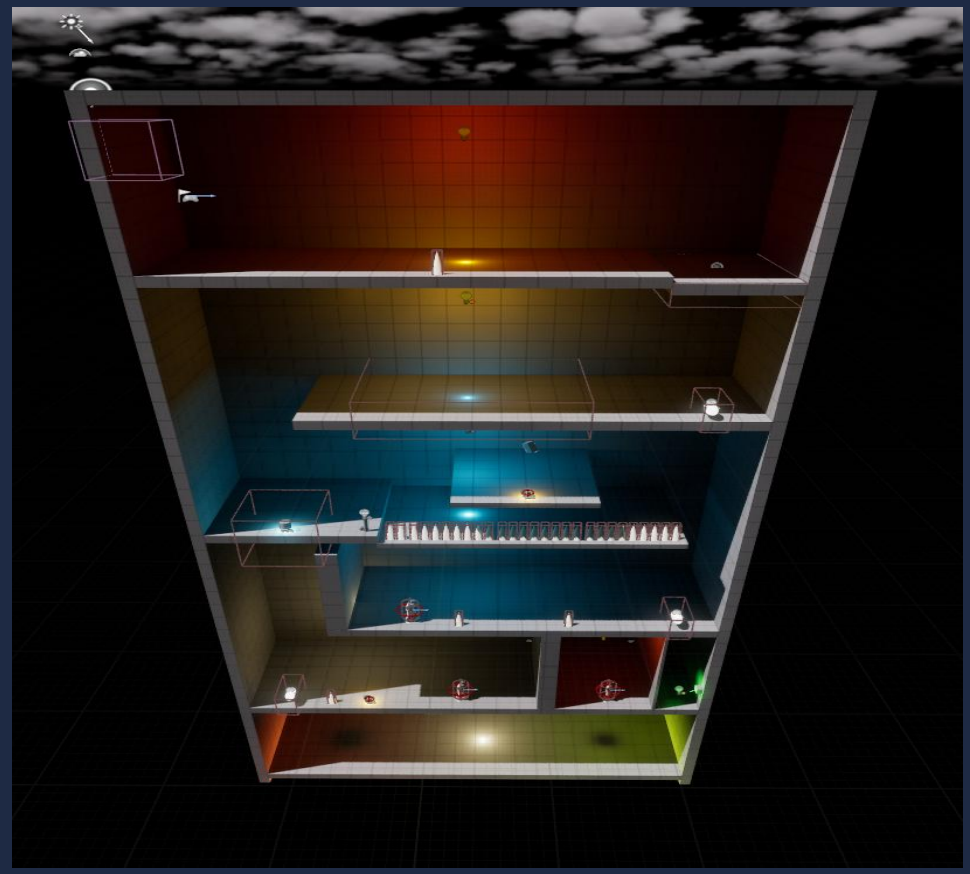
L1 Checkpoint, air pocket and a sliding floor you must slide across

L2 Torch and crystal reveal the bridge to the first Echo

L3 The Denizen drops the key for the locked door

L4 Spike trap, the second Echo, and a lever that opens the way down

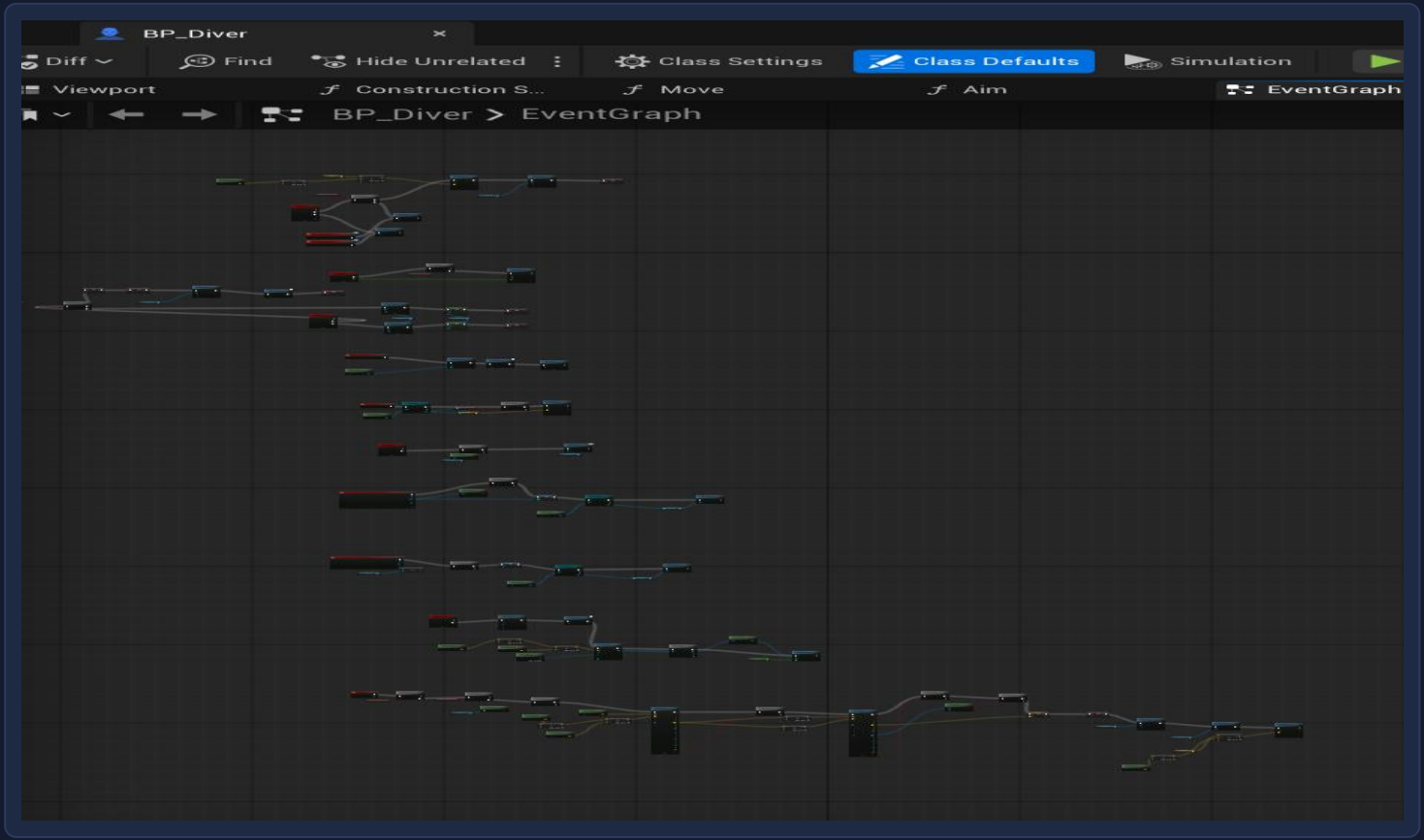
L5 Air pocket, the last Echo and the tool that ends the level



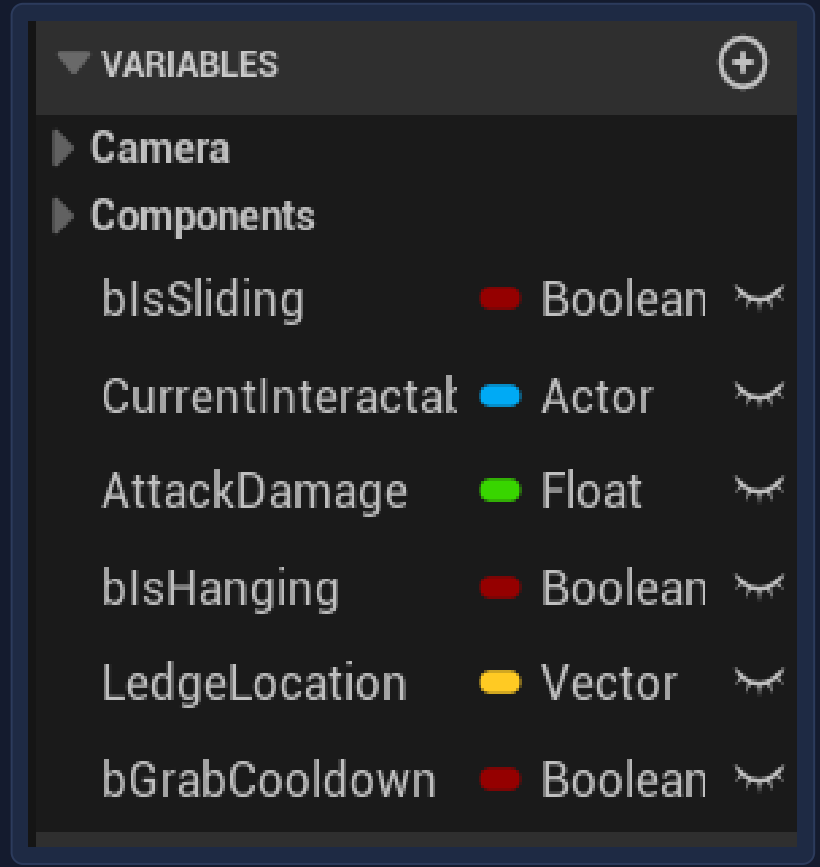
L_SurfaceRuins, seen from outside

The player

One Blueprint holds every player system, and every value the design needs to tune lives on it as a variable.



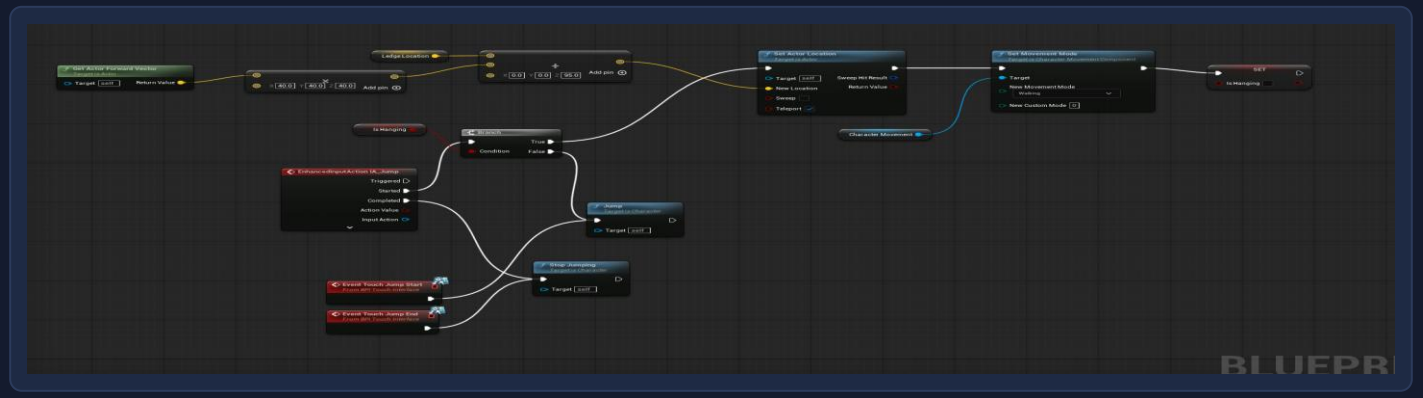
BP_Diver, Event Graph



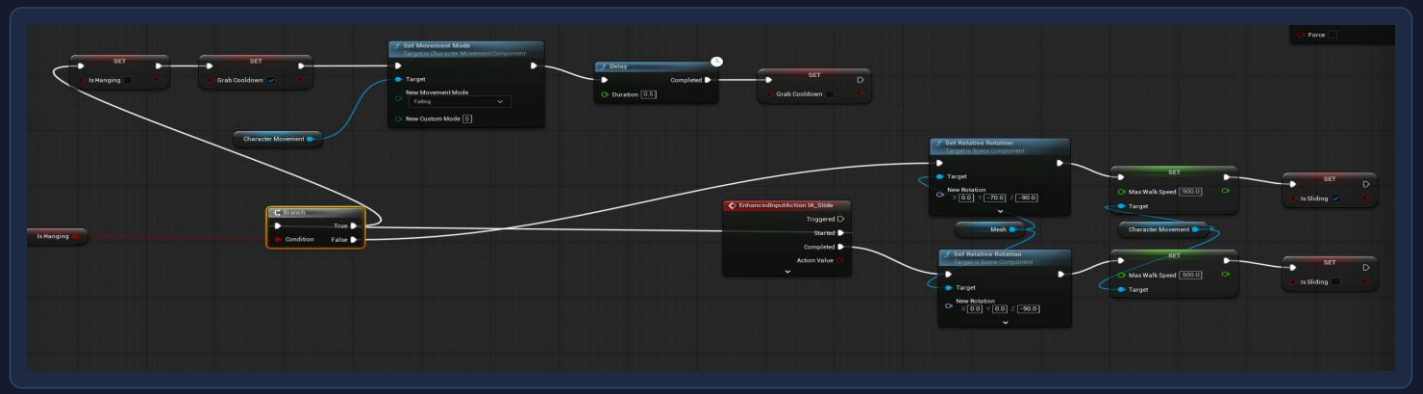
BP_Diver, variables

Jumping, sliding and dying

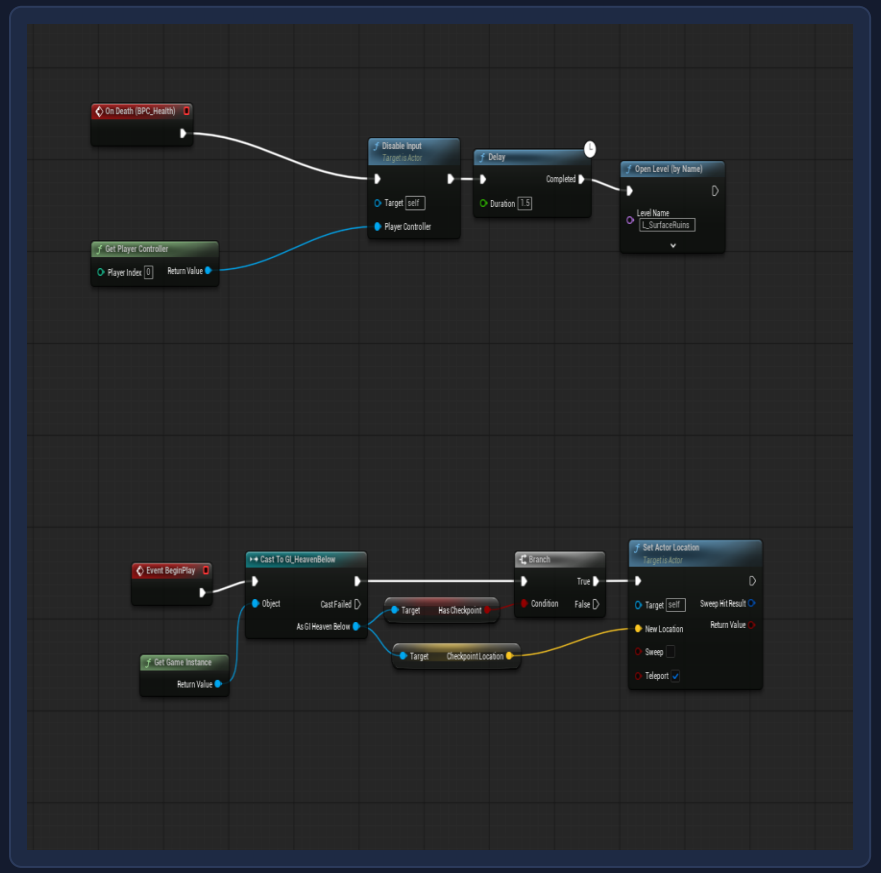
Jump either pulls the Diver up a held ledge or jumps normally, sliding tilts the mesh and raises walk speed, and death disables input then reloads the level at the last checkpoint.



BP_Diver, jump and mantle



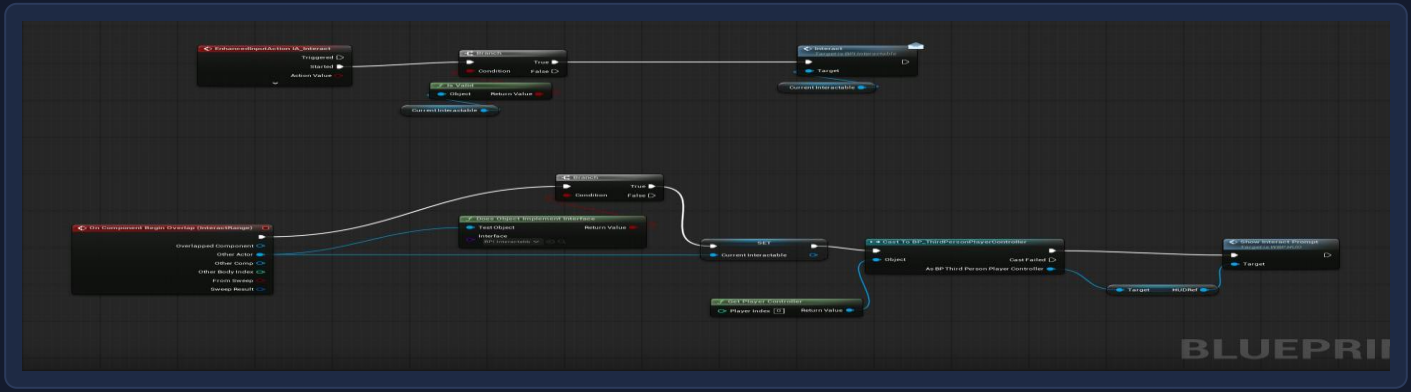
BP_Diver, slide



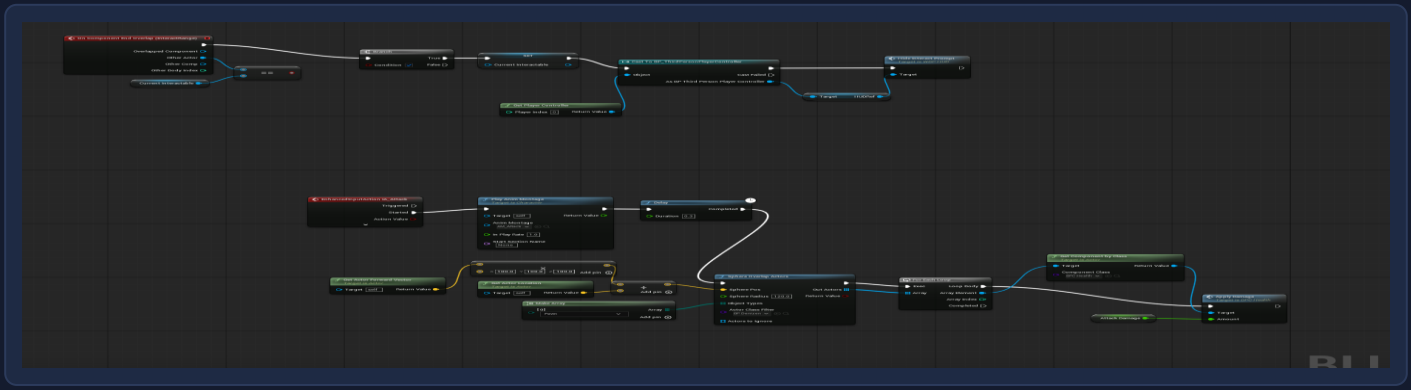
BP_Diver, death and respawn

Interacting, attacking and climbing

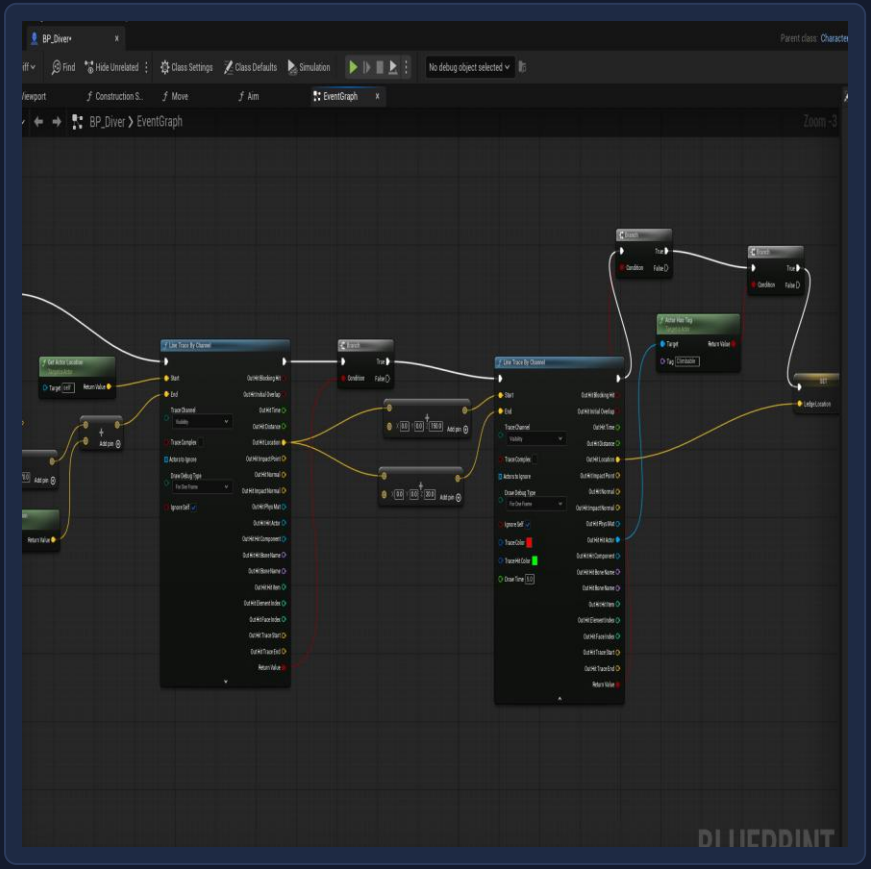
A sphere on the Diver watches for anything that implements the interact interface, the attack sweeps for Denizens in front of the player, and two line traces look for a ledge to grab while falling.



BP_Diver, interact



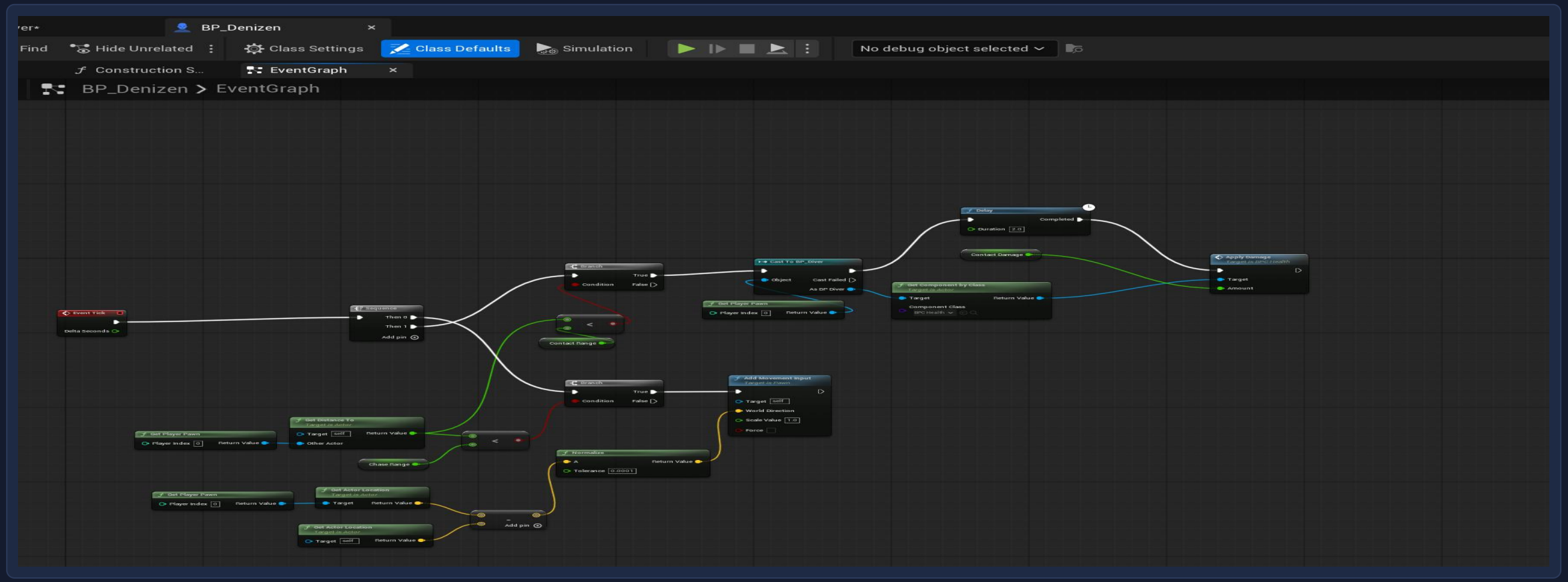
BP_Diver, attack



BP_Diver, ledge detection

The Denizen

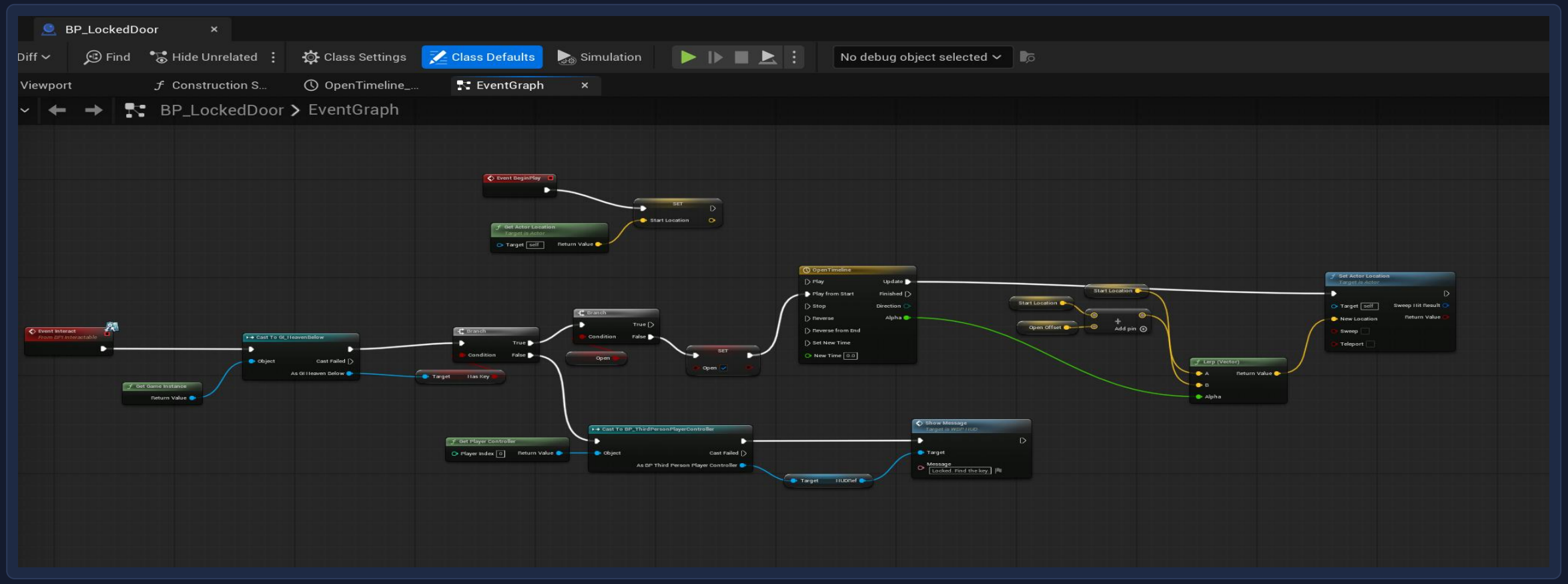
The Denizen measures its distance to the player every frame and either walks toward them or damages them on contact with a two second cooldown.



BP_Denizen, Event Graph

The locked door

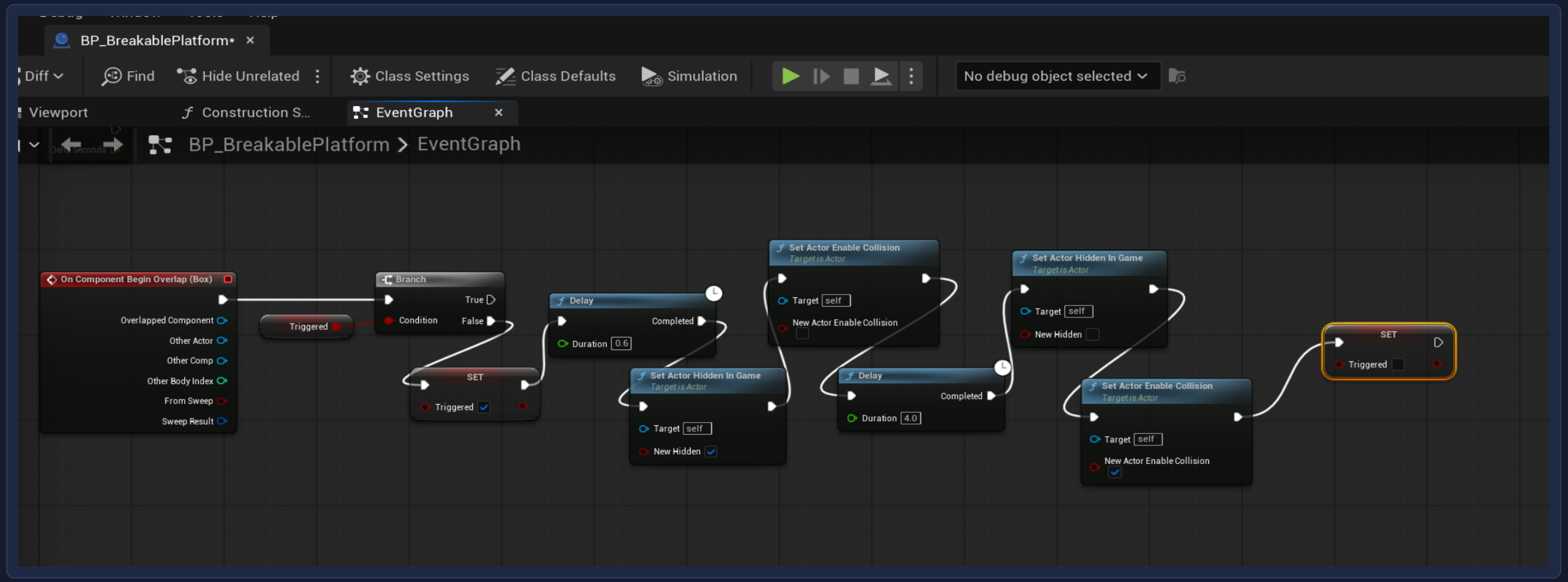
The door checks the Game Instance for the key and either opens on a Timeline or tells the player through the HUD that it is still locked.



BP_LockedDoor, Event Graph

The breakable platform

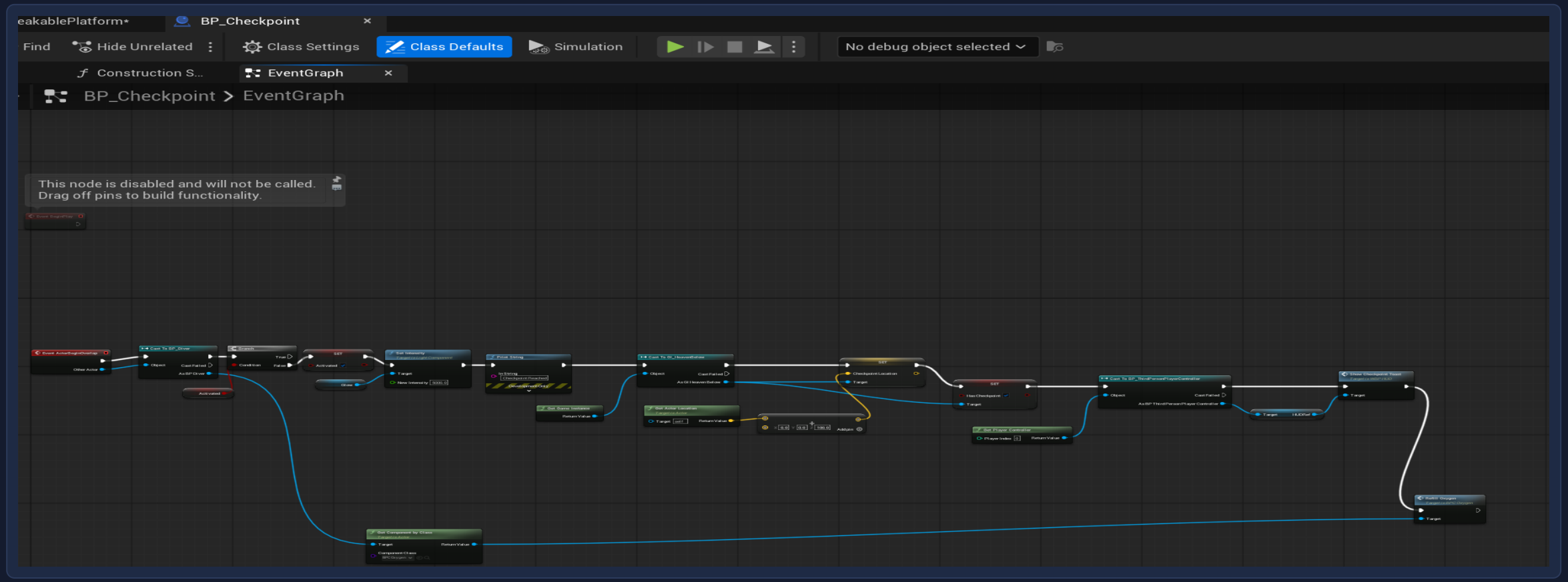
Standing on it starts a short delay, then it hides itself and drops its collision, and four seconds later it returns so the level can be replayed.



BP_BreakablePlatform, Event Graph

Checkpoints

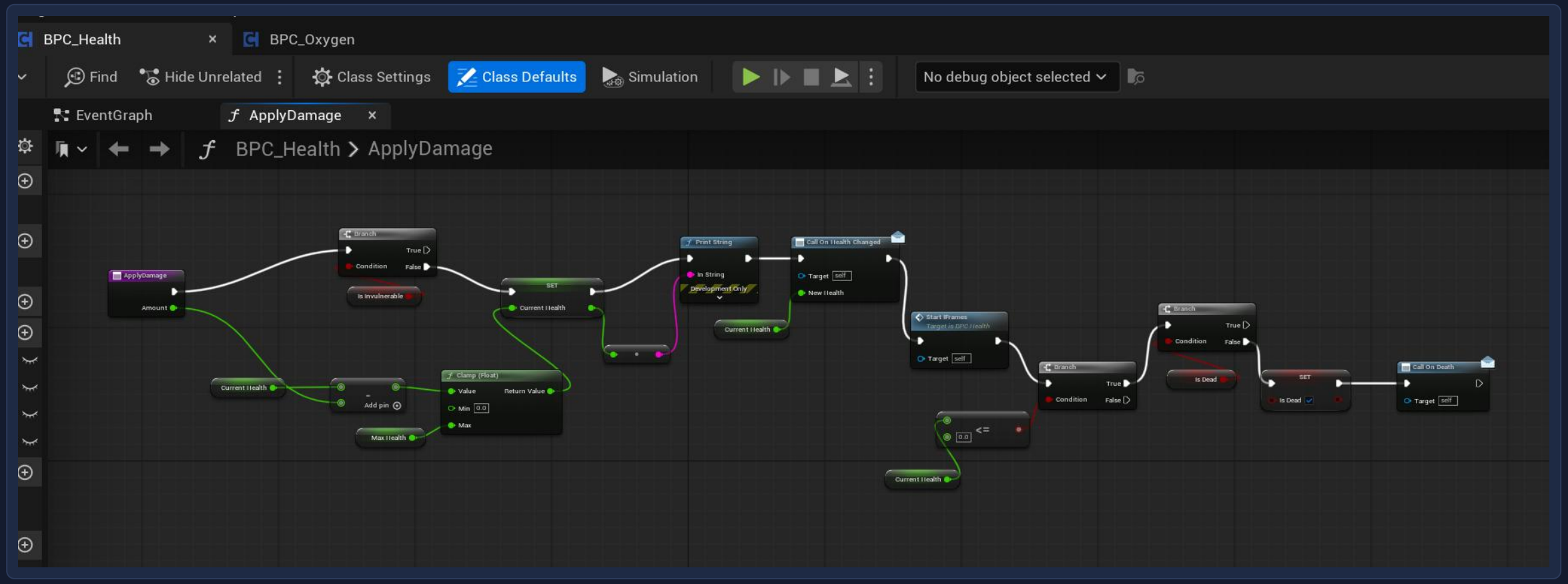
Touching a checkpoint lights it, stores the location on the Game Instance so it survives a level reload, refills oxygen and shows a toast on the HUD.



BP_Checkpoint, Event Graph

Shared health

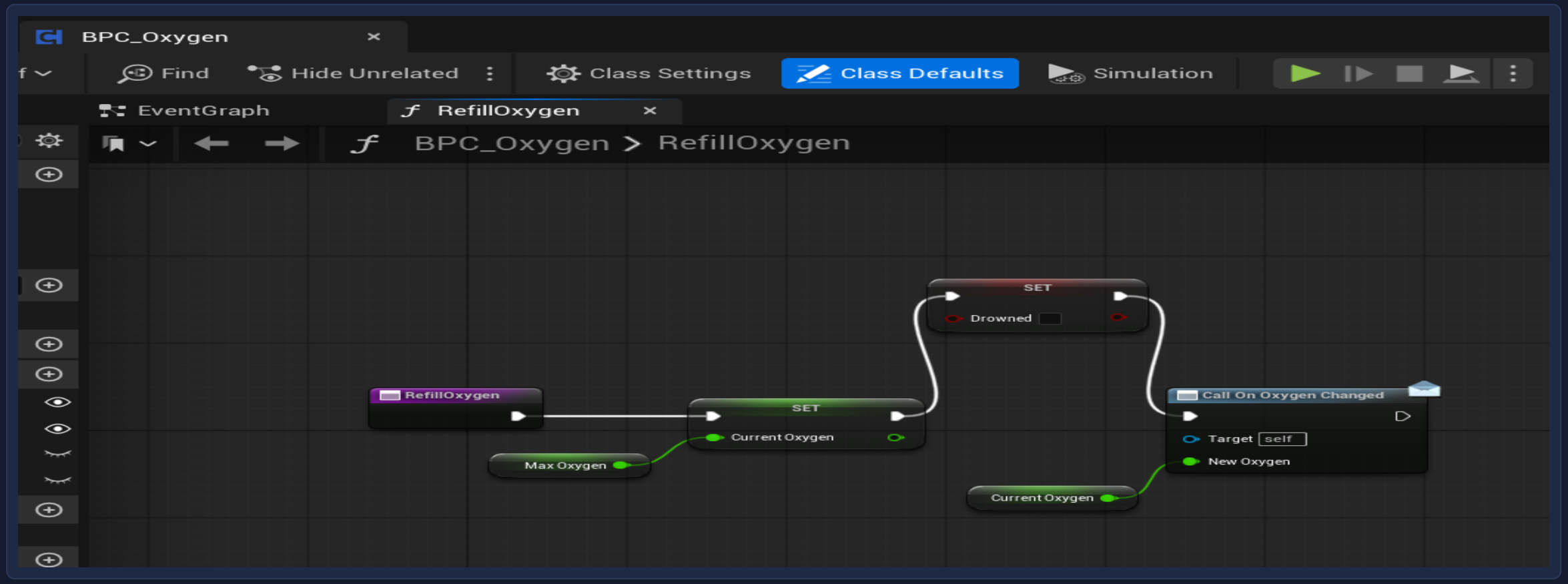
One reusable component handles damage for the Diver and the Denizen, clamping health, granting a second of invulnerability and firing death only once.



BPC_Health, ApplyDamage

Oxygen

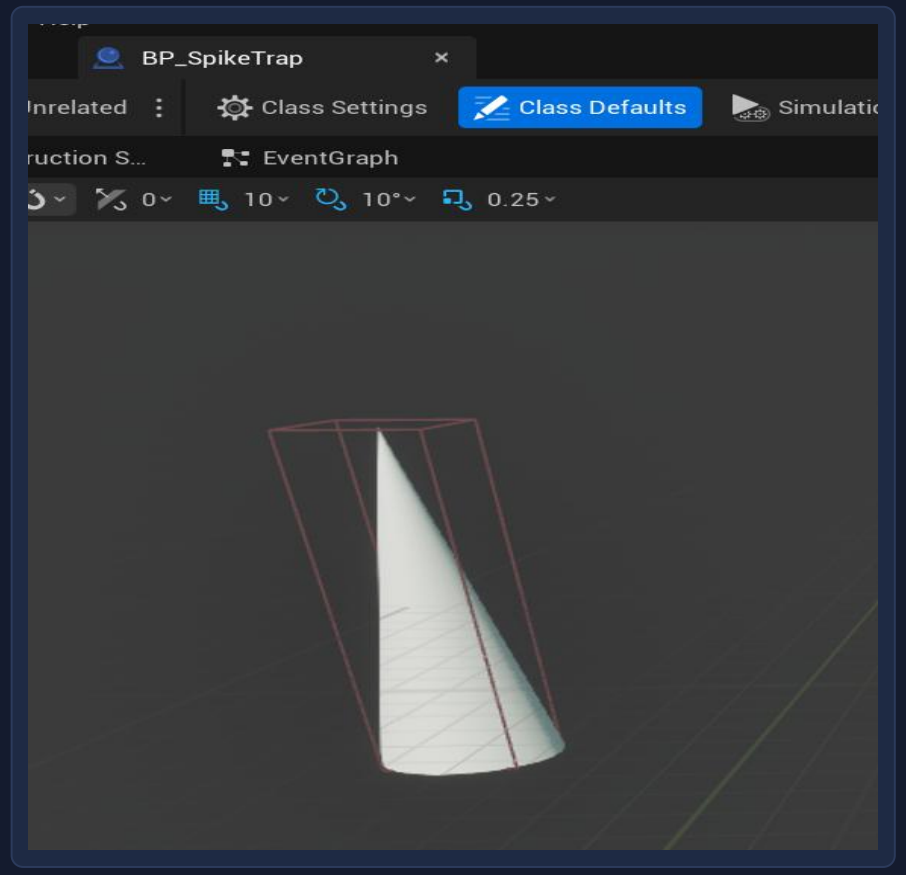
Oxygen drains on a timer and is the real clock on the level, so refilling it resets the value, clears the drowned flag and updates the HUD.



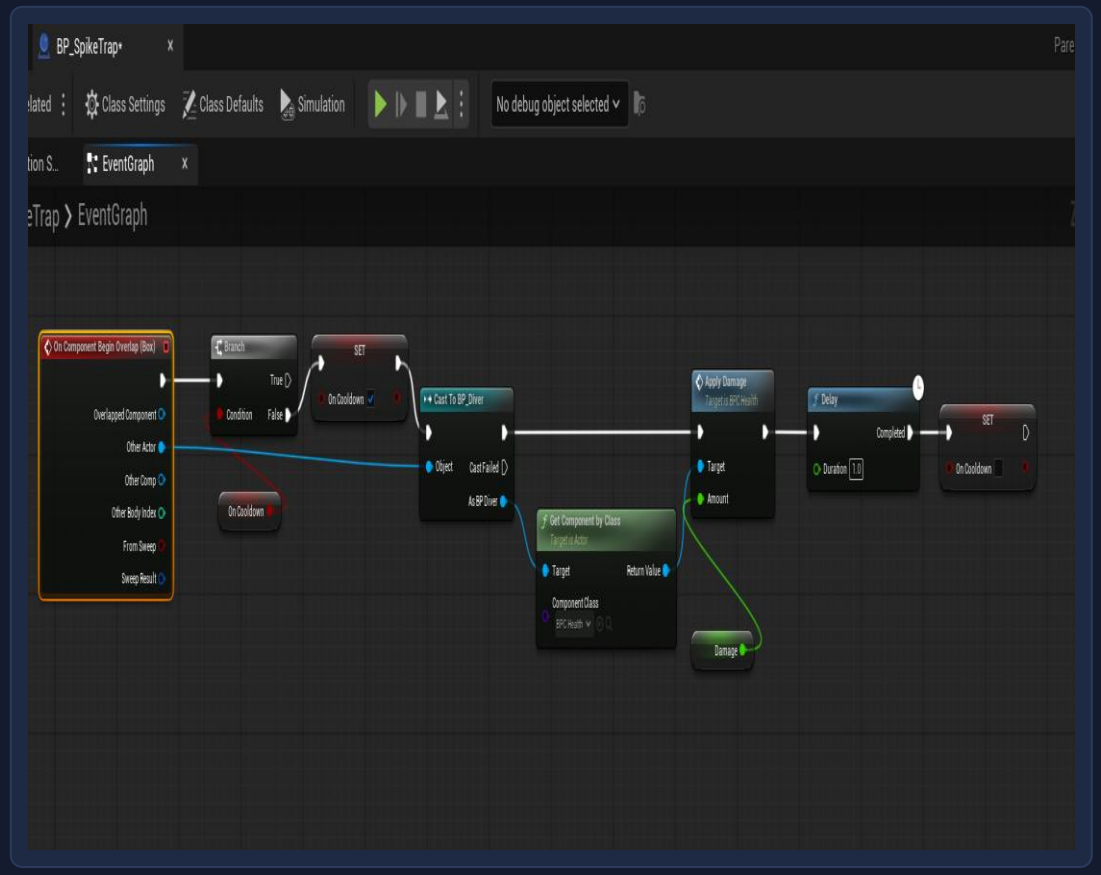
BPC_Oxygen, RefillOxygen

The spike trap

The trigger box is deliberately larger than the spike mesh so the hazard is not missed at running speed, and a one second cooldown stops it draining health every frame.



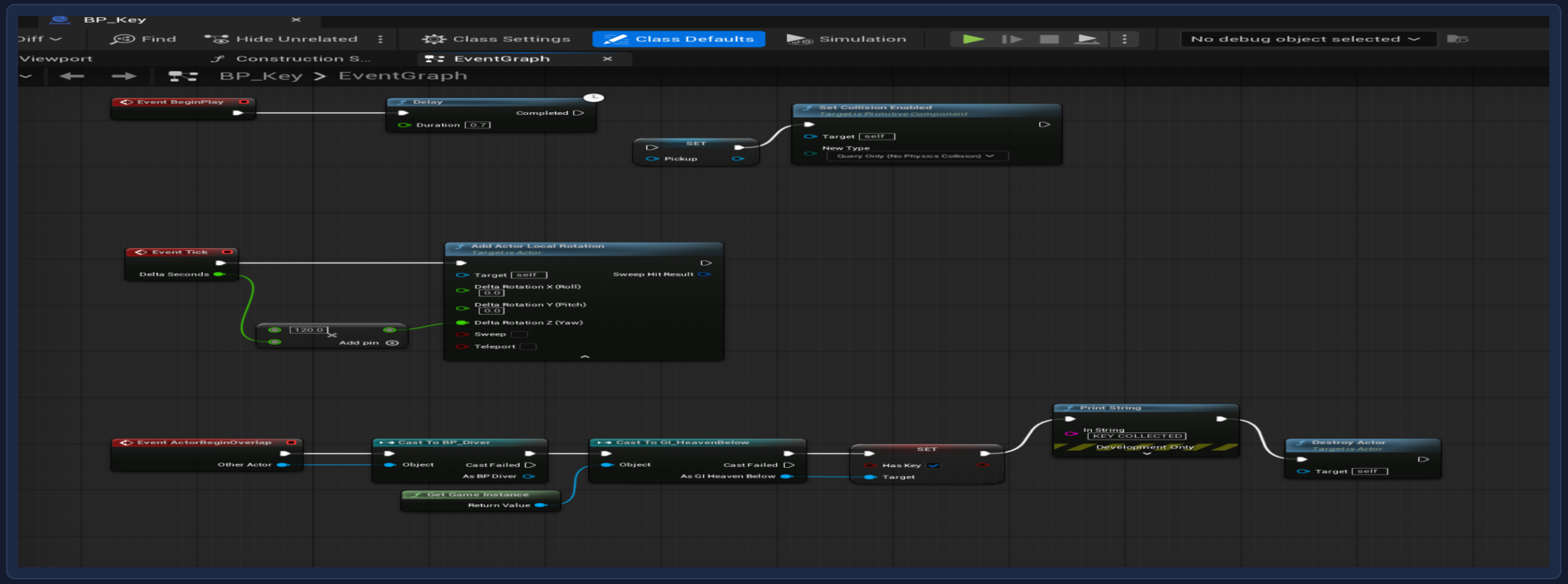
BP_SpikeTrap, viewport



BP_SpikeTrap, Event Graph

The key

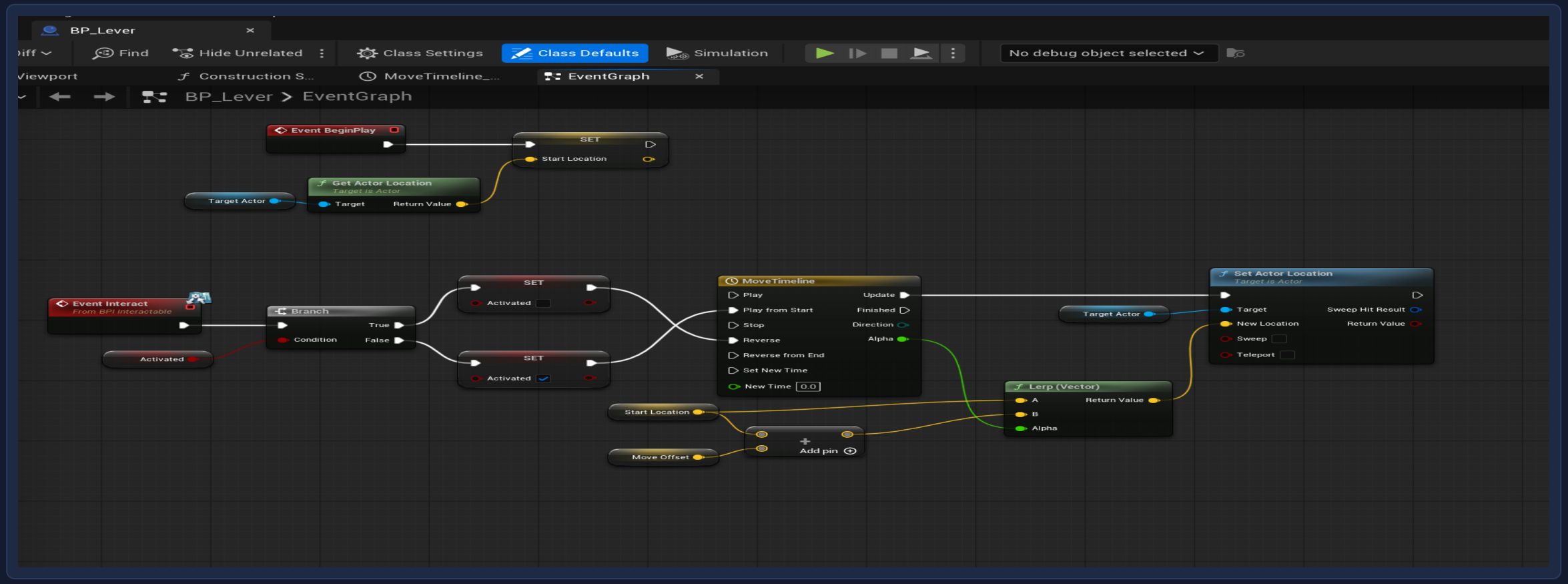
The key spins to draw the eye, arms its pickup collision after a short delay so it cannot be collected inside the Denizen, and records itself on the Game Instance.



BP_Key, Event Graph

The lever

The lever stores its target's starting position on begin play, then drives a Timeline that moves that actor by an offset set on each copy.



BP_Lever, Event Graph

What I learned

Scope honestly

The design document describes five biomes and a boss. I built one biome and recorded every cut and the reason for it rather than pretending the rest was still coming.

Components beat copies

Writing health once as a component meant the Diver and the Denizen shared it, so adding a second damageable actor cost minutes instead of hours.

Geometry can lie

An engine cube ships with a collision box over three times the size of its mesh. It read as floating above floors and stopping short of walls until I compared the two numbers directly.

Read the asset, not the symptom

Most of my worst bugs produced no error at all. Checking the values actually stored in the asset found in minutes what guessing had not found in hours.

GUNSMOKE MINES

Development Log

Bruno Mirone FGCT4017 Fundamentals of Games Design BA (Hons) Game Design, Level 4, UCA

Built in Unreal Engine 5.6 using Blueprints only

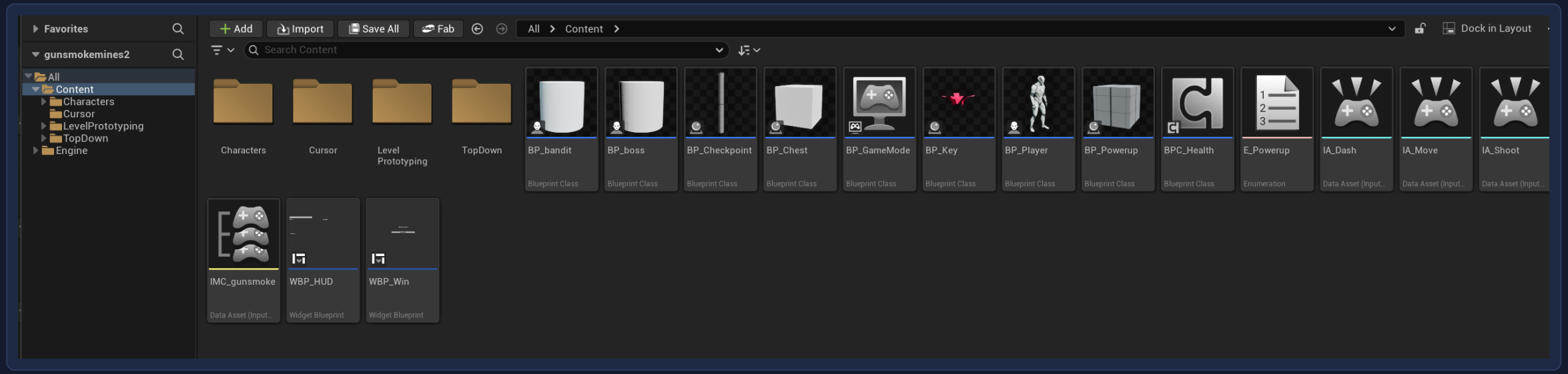
The level

The mine is one continuous level of connected rooms and corridors, blocked out from cubes.

L_Mine, seen from above

Everything in the project

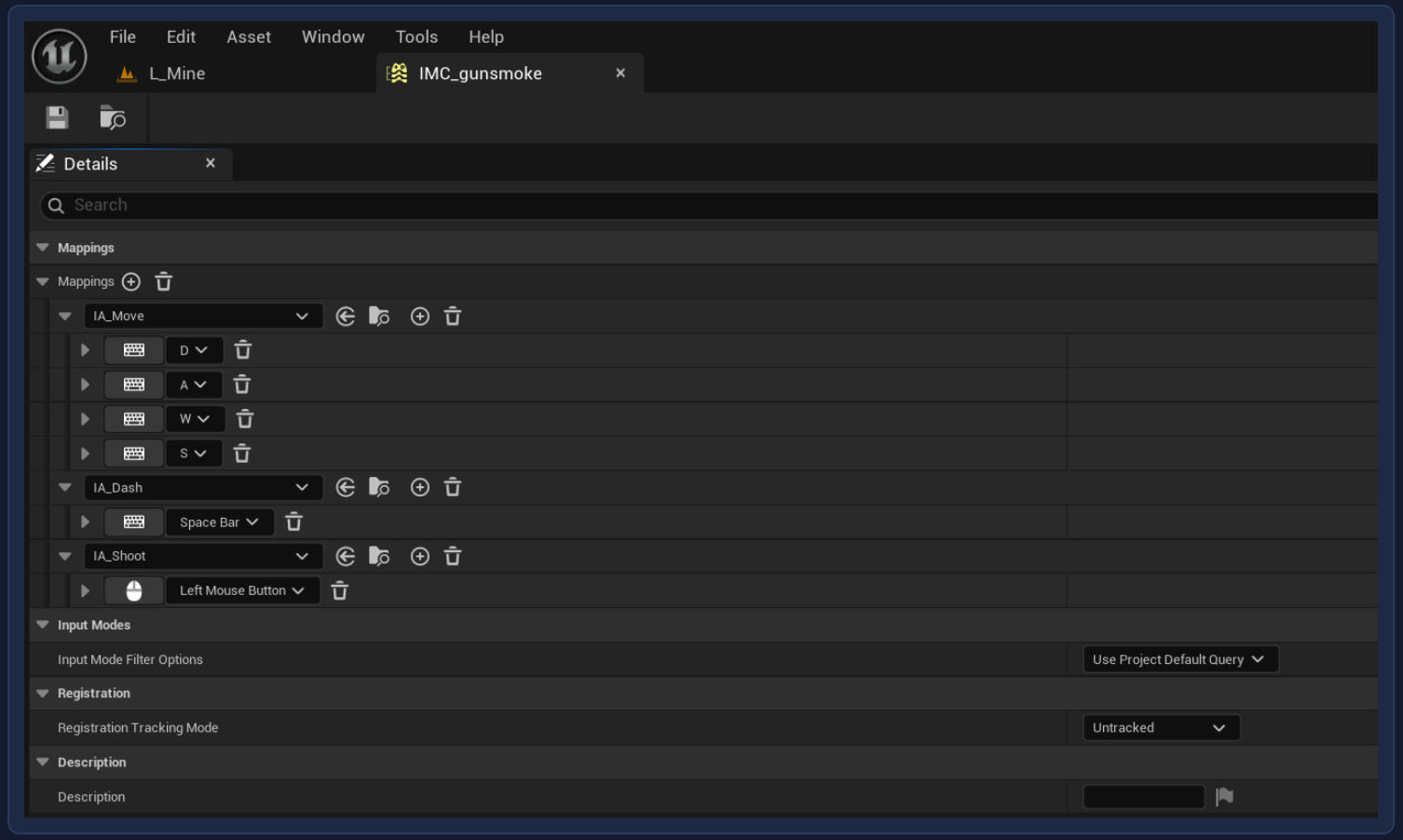
Seventeen assets in total: eight Blueprints, a shared component, two widgets, the input set and one level.



Content Browser

Input

Enhanced Input maps WASD, Space and the left mouse button through a single mapping context.

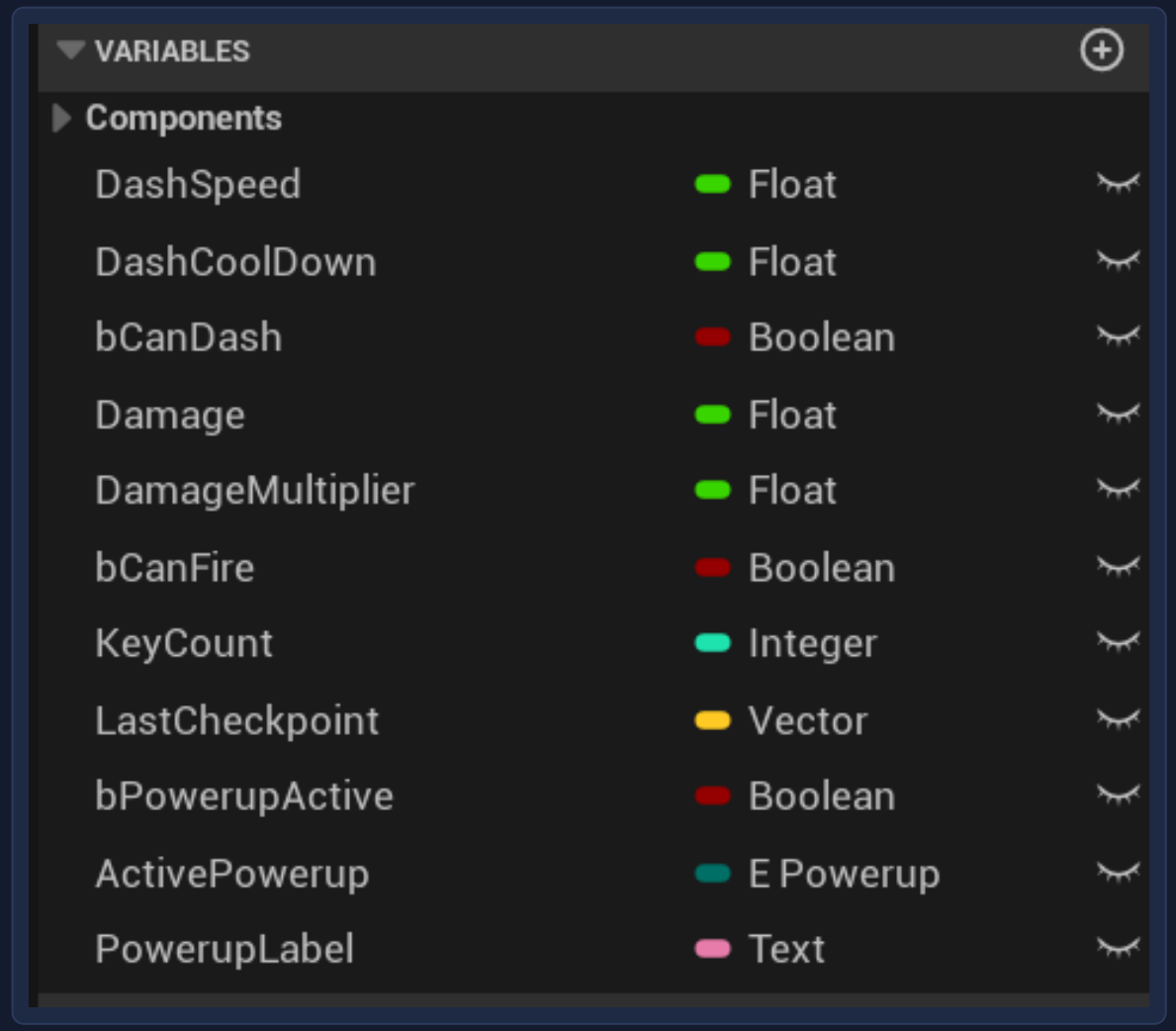


IMC_gunsmoke

Player variables

Every tunable value lives on the player as a variable, so balancing means editing numbers rather than rewiring.

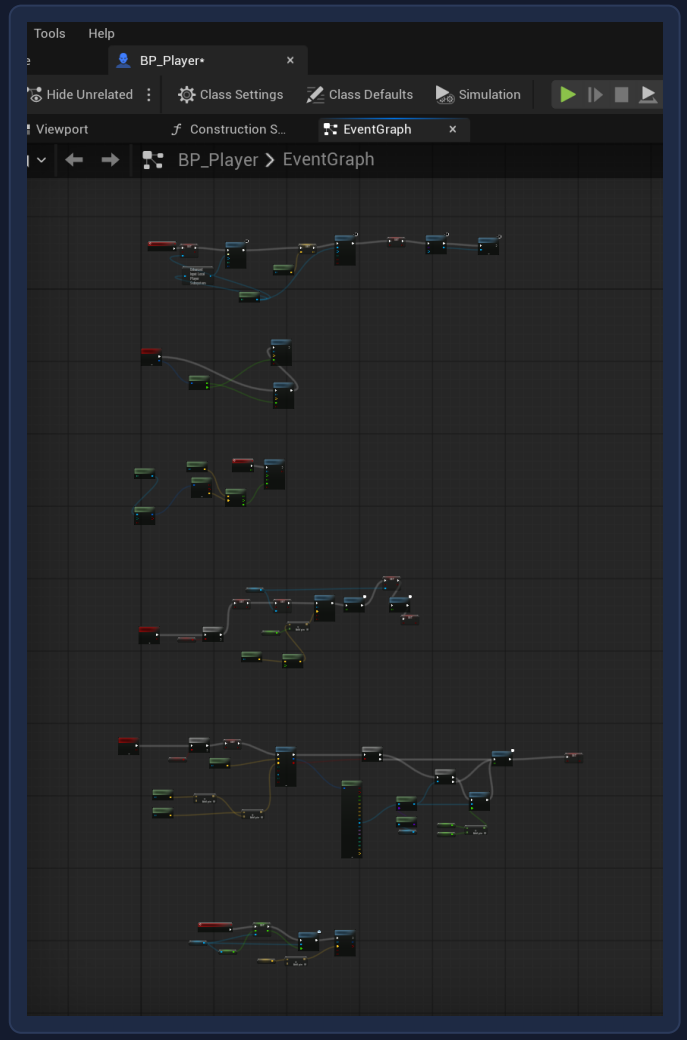
BP_Player, variables



The player

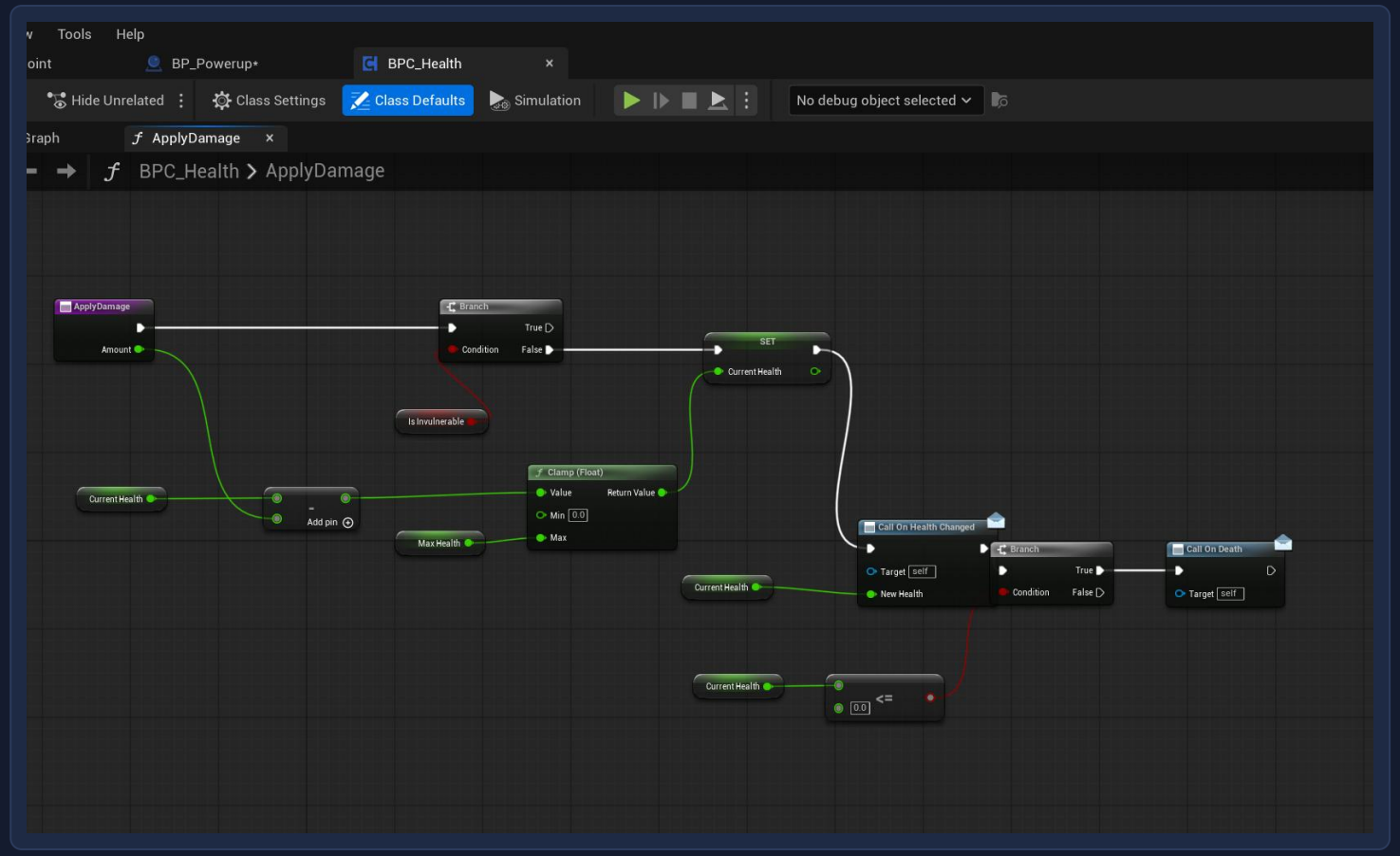
One graph handles movement, mouse aim, the dash, shooting and respawning at the last checkpoint.

BP_Player, Event Graph



Shared health

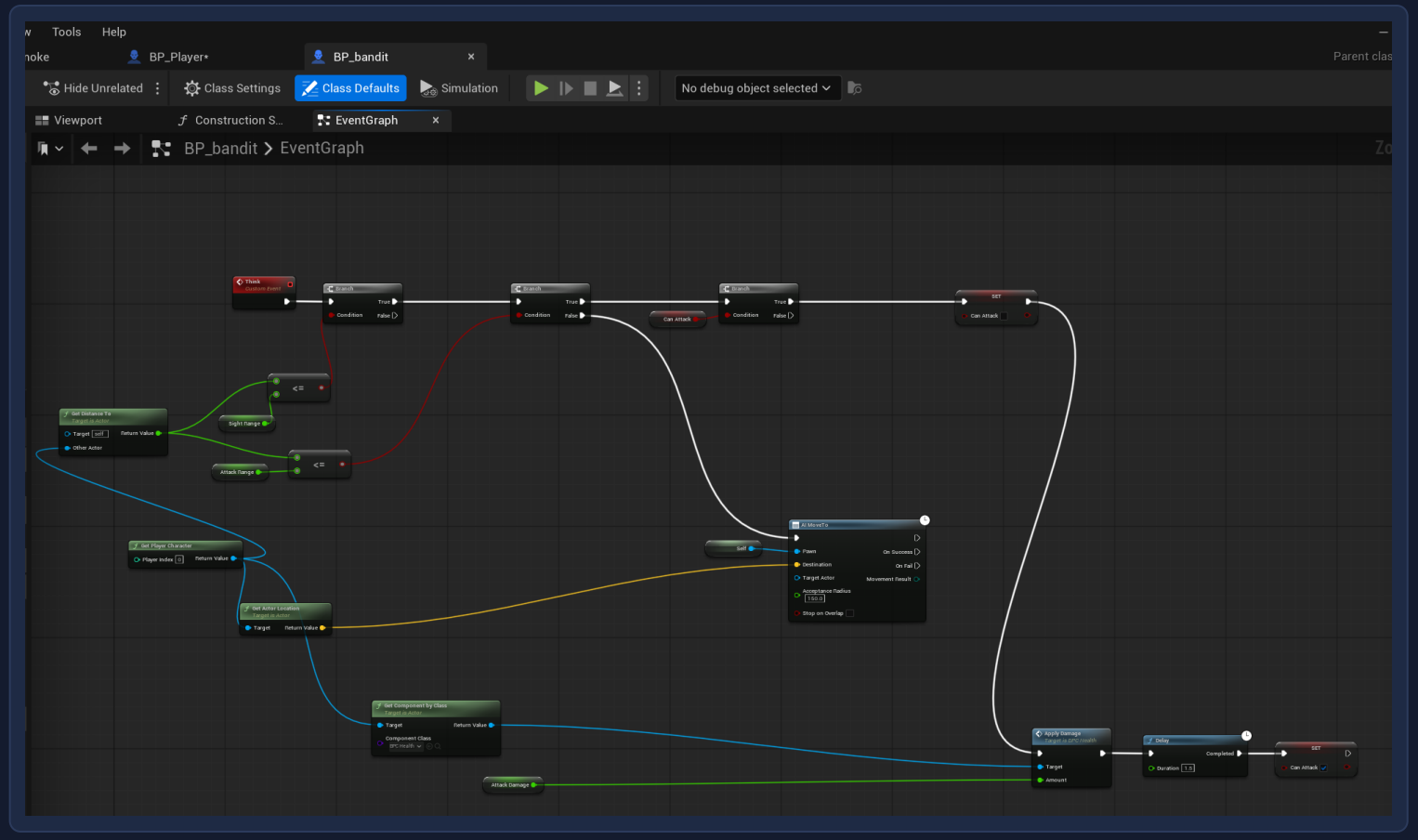
A single reusable component handles damage, death and health changes for the player, the bandits and the boss.



BPC_Health, ApplyDamage

Bandit AI

The bandit measures its distance to the player four times a second and either closes in or attacks.

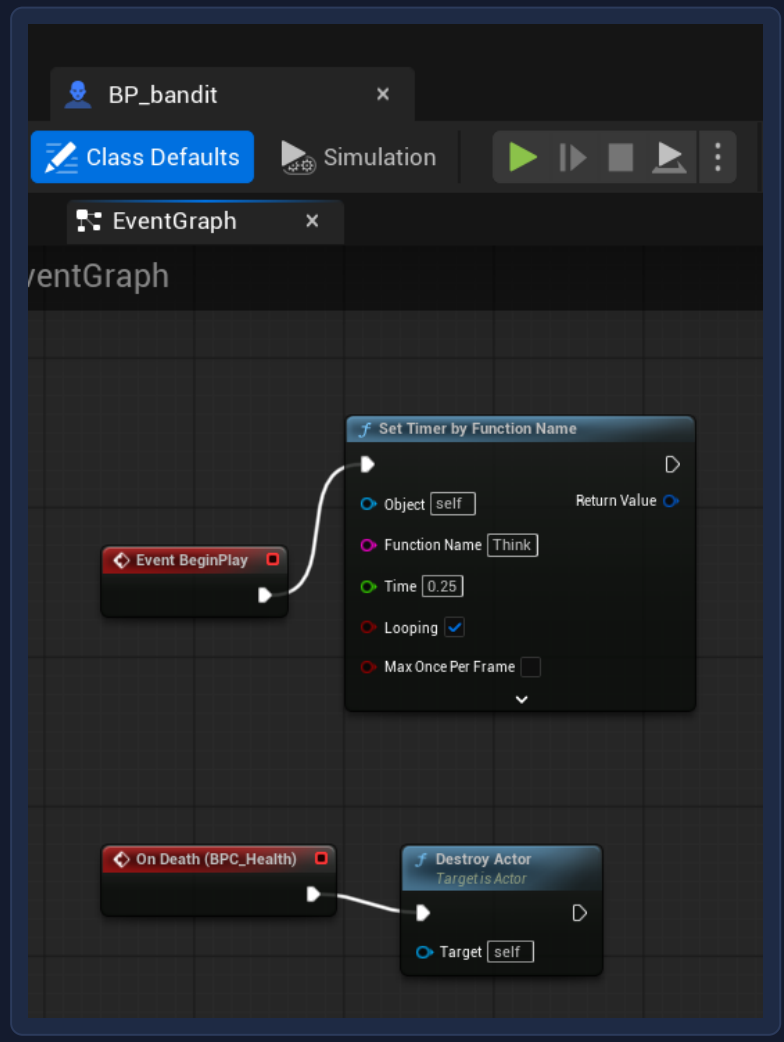


BP_bandit, Event Graph

Bandit lifecycle

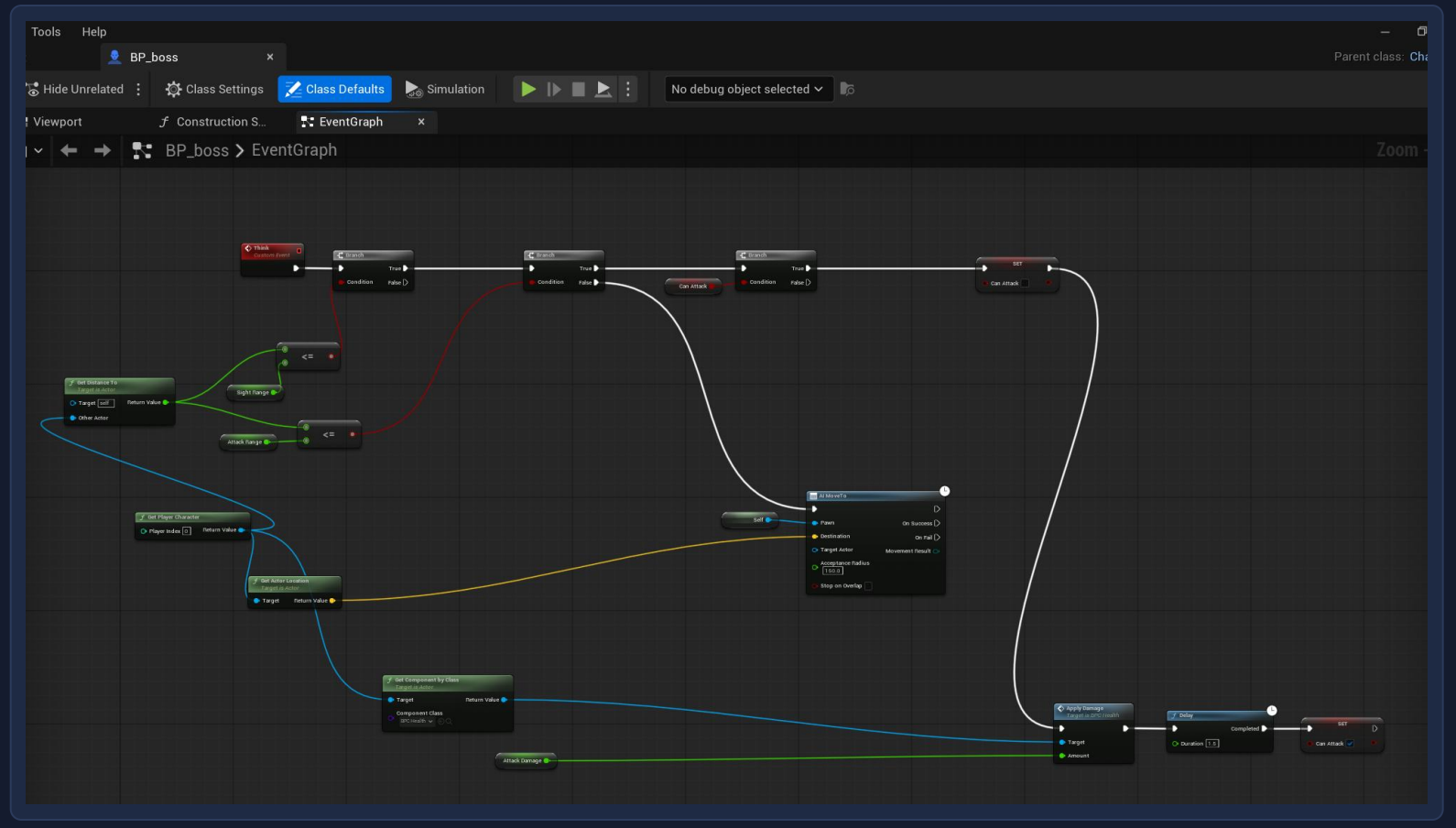
A looping timer drives the AI, and the health component's death event destroys the actor.

BP_bandit, BeginPlay and death



The boss

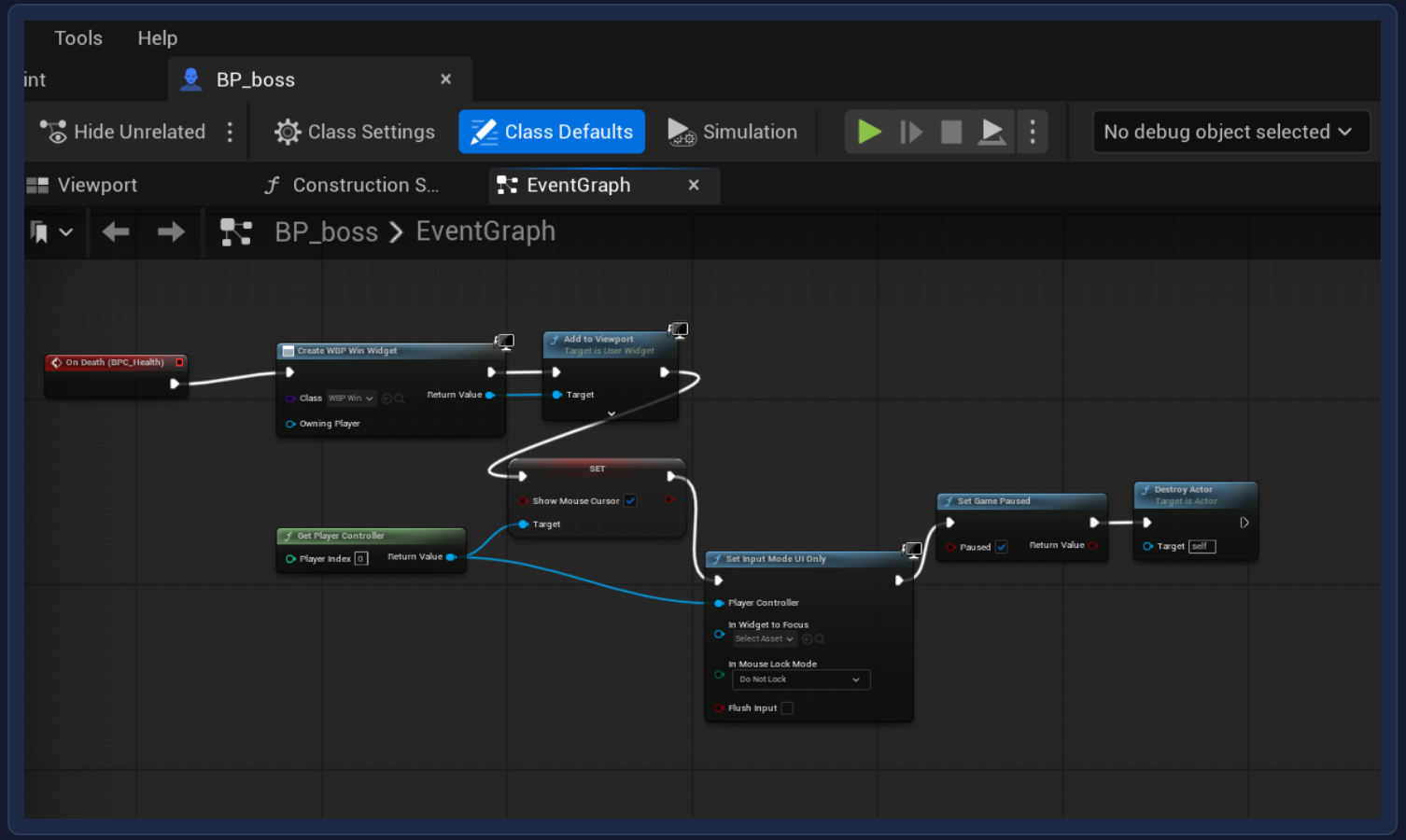
The boss reuses the bandit's logic with a much larger health pool, slower movement and heavier hits.



BP_boss, Event Graph

Winning

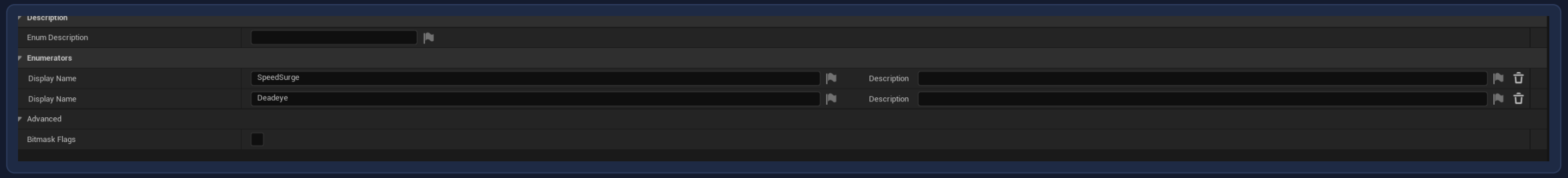
Killing the boss creates the win widget, pauses the game and hands control back to the mouse.



BP_boss, win condition

Two kinds of powerup

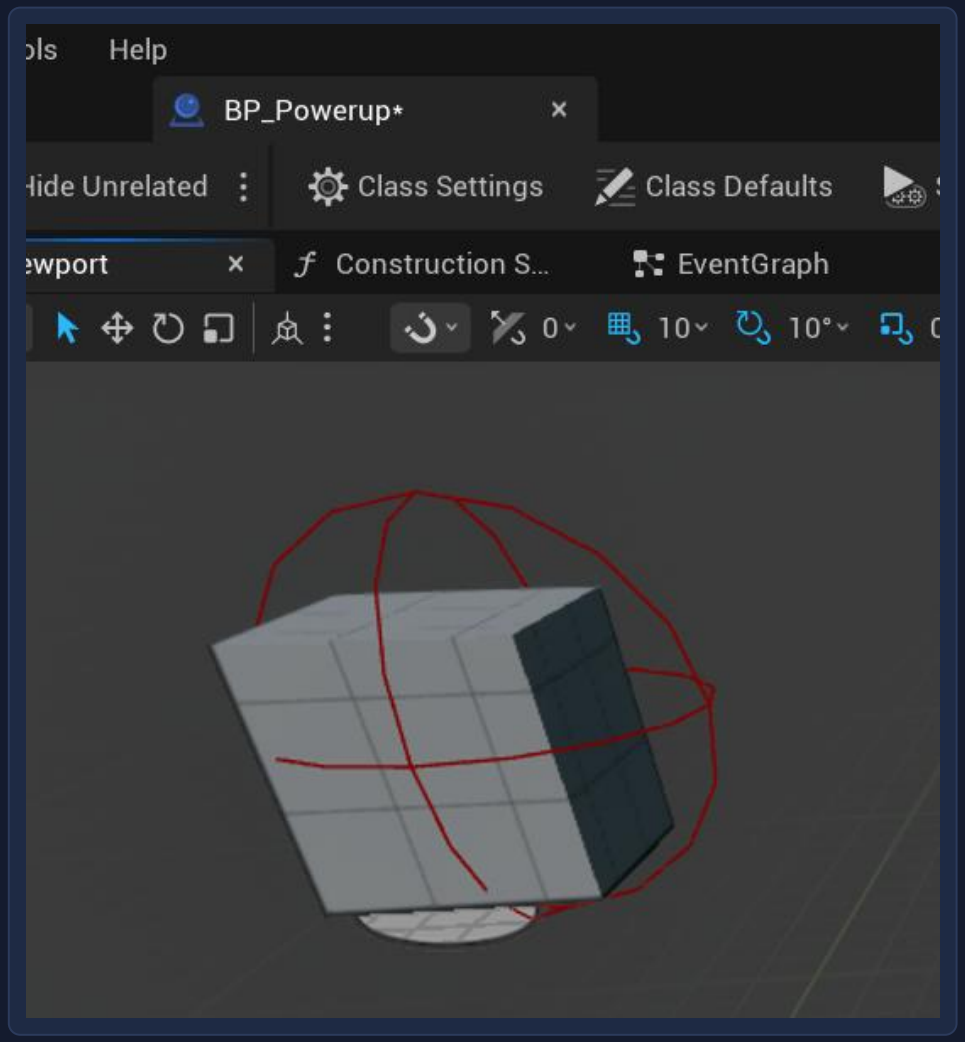
An enumeration keeps the two powerup types readable in the graph instead of using numbers.



E_Powerup

One Blueprint, both powerups

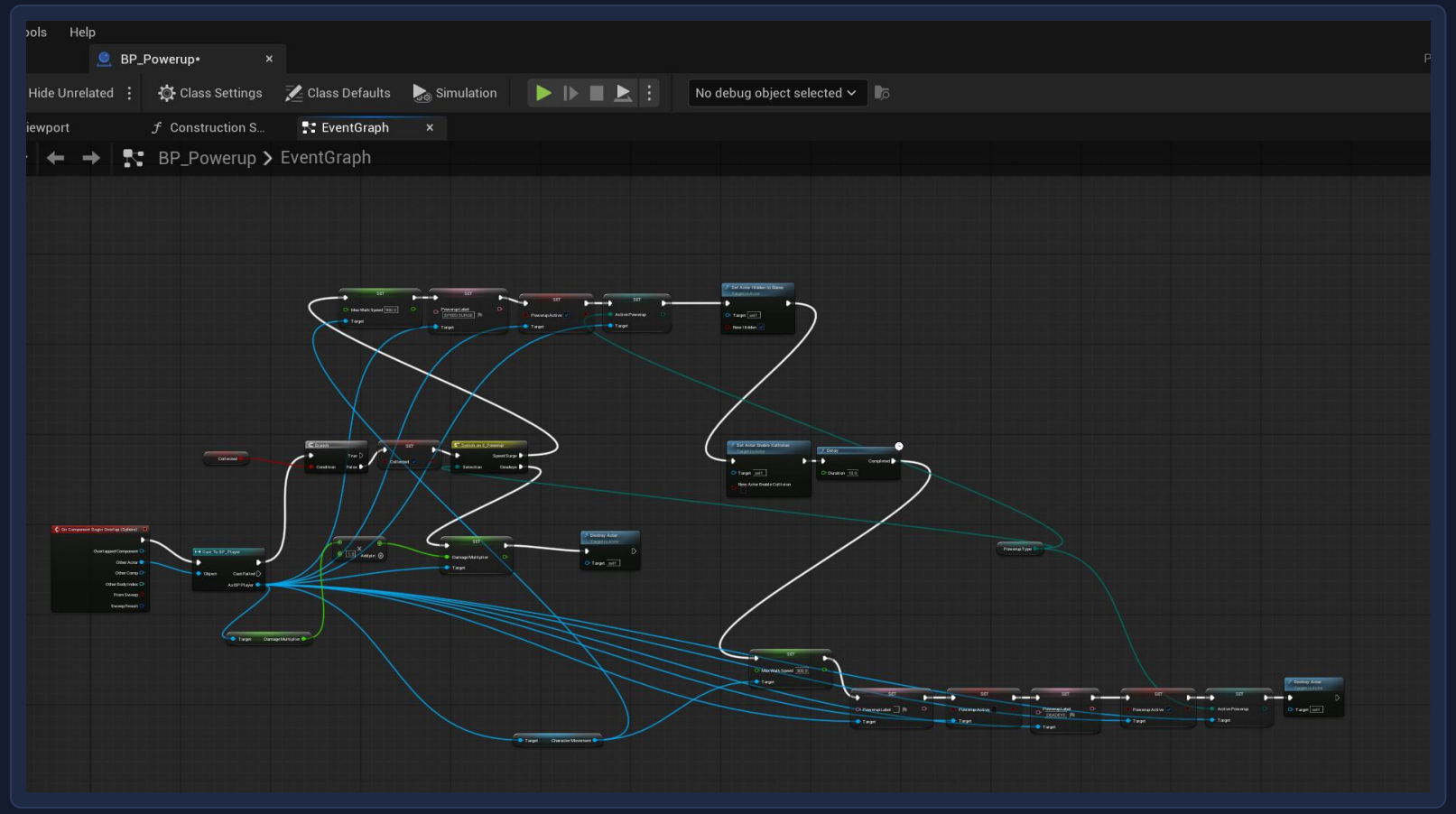
A single actor covers both types, chosen per copy placed in the level.



BP_Powerup, viewport

Powerup logic

A switch splits into a timed speed boost that reverts itself, and a permanent damage increase.

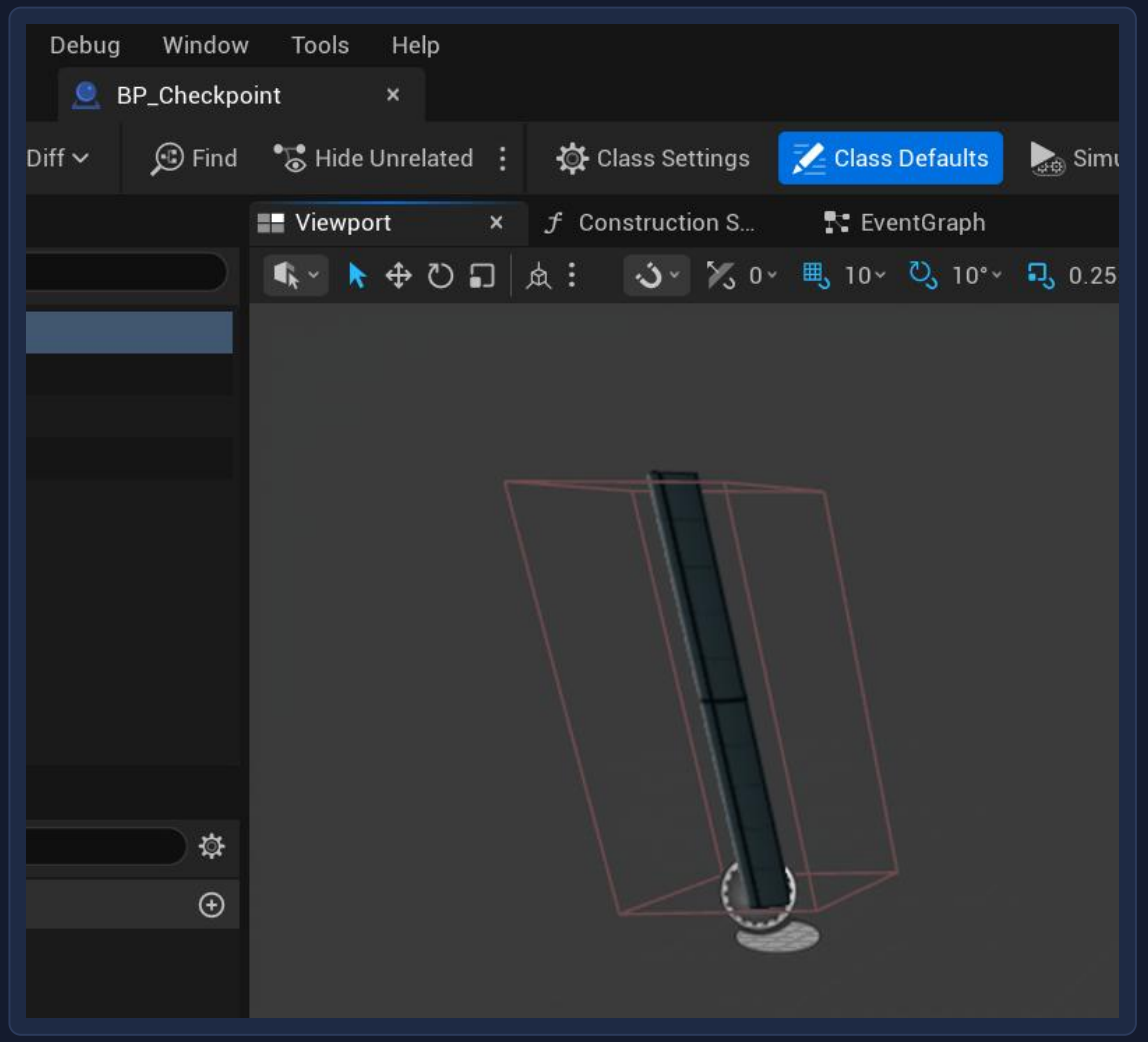


BP_Powerup, Event Graph

Checkpoints

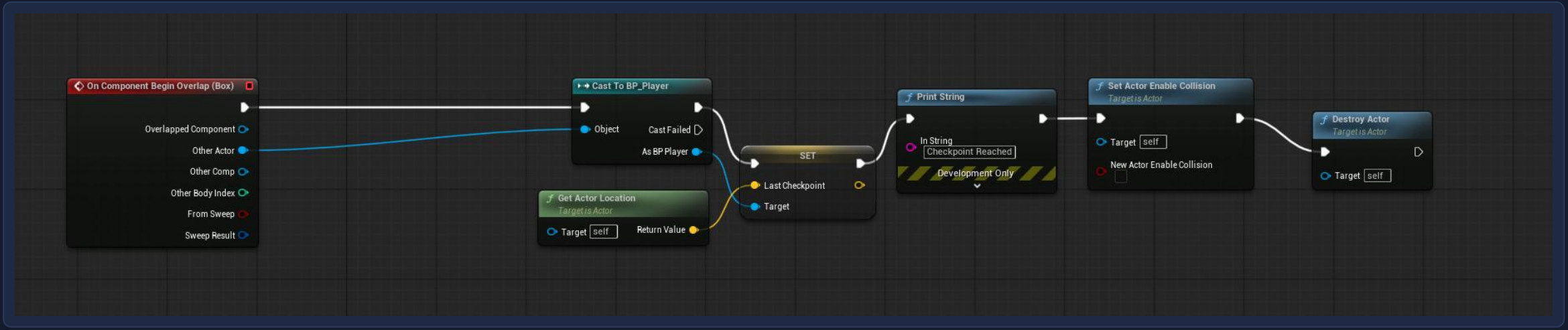
The checkpoint is a post with a box collision deliberately larger than its mesh, so it is hard to miss.

BP_Checkpoint, viewport



Saving your place

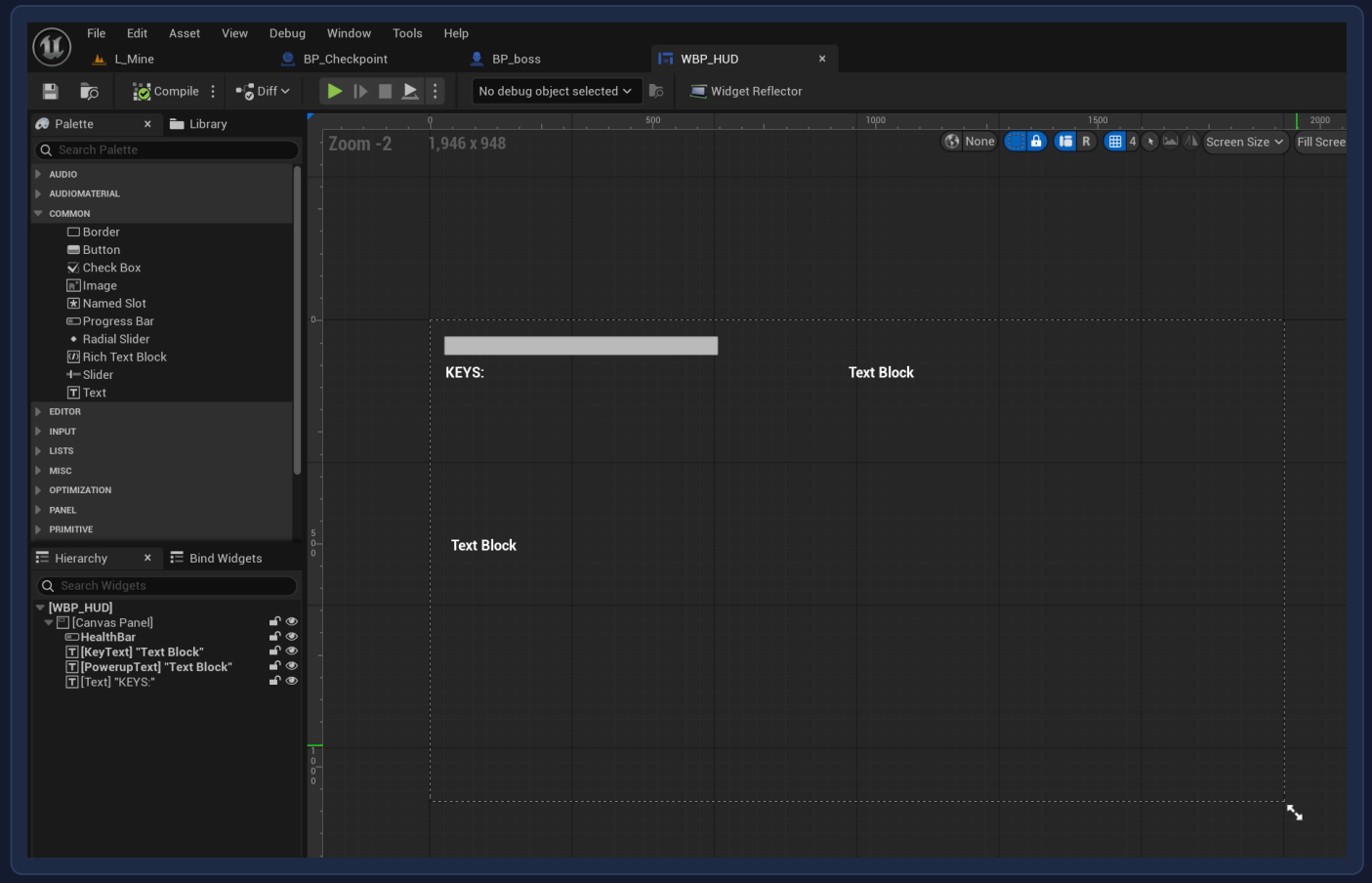
Touching it stores the location on the player and switches off its own collision so it only fires once.



BP_Checkpoint, Event Graph

The HUD

The interface is deliberately minimal: a health bar, a key counter and a powerup label.

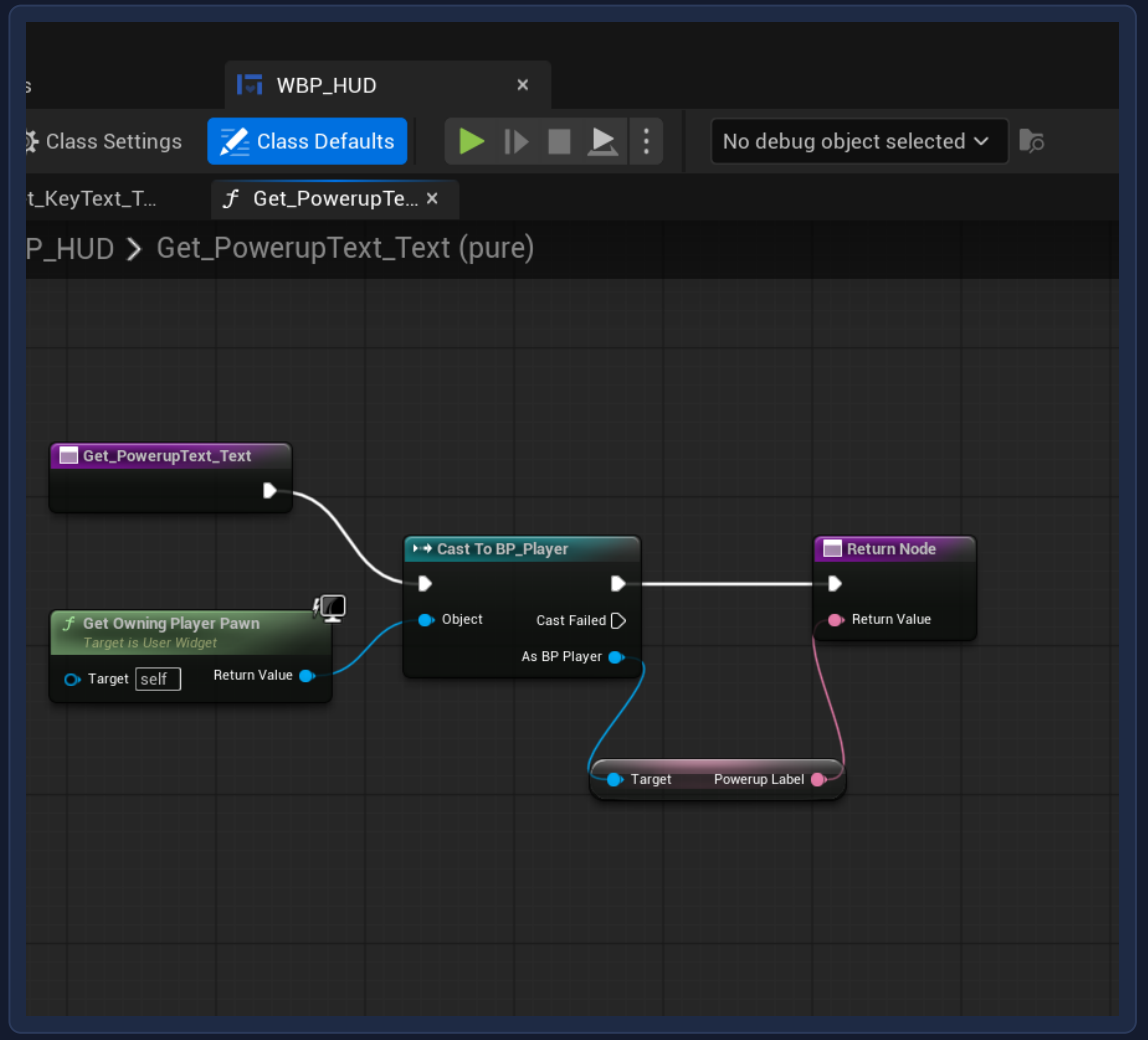


WBP_HUD, designer

Reading the player

Each HUD element pulls its value from the player through a bound function that runs every frame.

WBP_HUD, text binding



What I learned

Scope first

The design document describes seven weapons and eight powerups. I built one weapon and two powerups, and recorded every cut and the reason for it.

Reuse beats repetition

Writing the health system once as a component meant the player, the bandits and the boss all shared it, and the boss took minutes rather than hours.

Unreal fails quietly

Most bugs produced no error at all. A pawn ignoring the visibility channel, a node defaulting its target to itself, a mesh whose collision was larger than the mesh.

Debugging is a method

Print strings to split a problem in half, collision view to see the invisible, and checking one thing at a time. That mattered more than knowing the engine.